

บทที่ 6

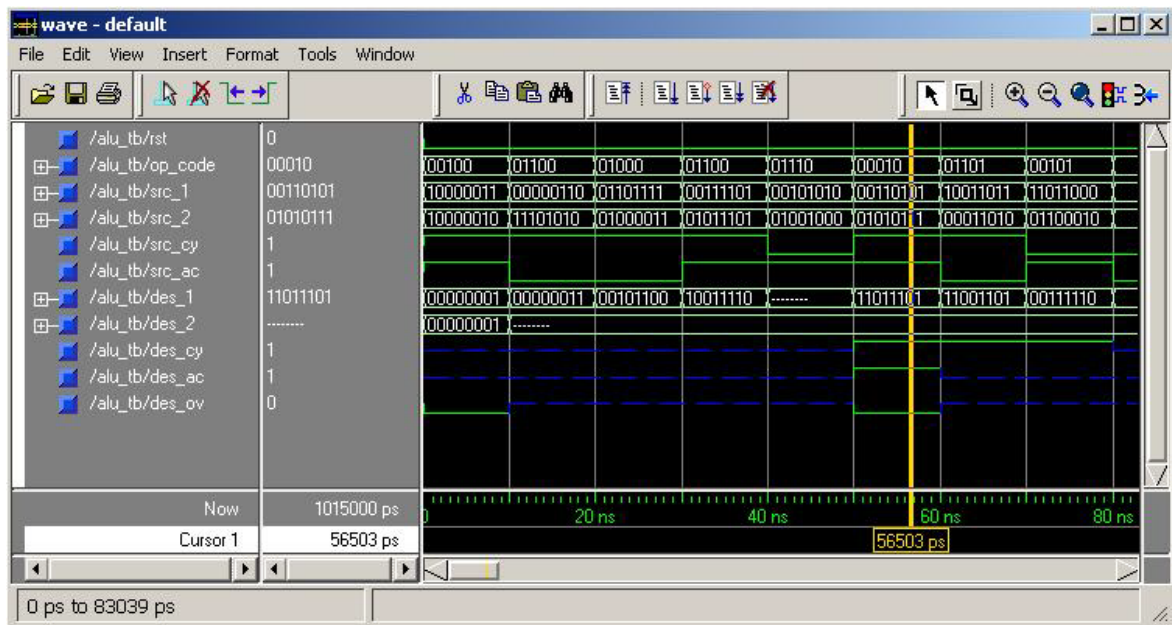
การทดสอบการทำงาน

6.1 การทดสอบการทำงานของไมโครคอนโทรลเลอร์

ในการทดสอบความถูกต้องของการทำงานของไมโครคอนโทรลเลอร์ในโครงการนี้ ใช้วิธีการจำลองการทำงาน (Simulation) ด้วยโปรแกรมจำลองการทำงาน ซึ่งเป็นวิธีตรวจสอบความถูกต้องของการทำงานที่มีประสิทธิภาพ สะดวกและมีค่าใช้จ่ายน้อยที่สุด โดยทำการเขียนไฟล์ทดสอบ Test Bench สำหรับทดสอบแต่ละส่วนของวงจรที่ออกแบบไว้ขึ้นมา เพื่อทำการป้อนสัญญาณทดสอบแล้วดูผลการตอบสนองการทำงานของวงจรที่แสดงออกมาในรูปของ Waveform โดยแบ่งการทดสอบออกเป็น 3 ระดับ

6.1.1 การทดสอบในระดับวงจรย่อย

เมื่อทำการเขียนวงจรย่อยๆในแต่ละตัวแล้วก็จะทำการทดสอบการทำงานของวงจรย่อยนั้น เช่น วงจรจัดลำดับคำสั่ง, วงจรถอดรหัสคำสั่ง, ALU, หน่วยความจำ เป็นต้น จากตัวอย่างรูปที่ 6.1 เป็นการทดสอบการทำงานของ ALU

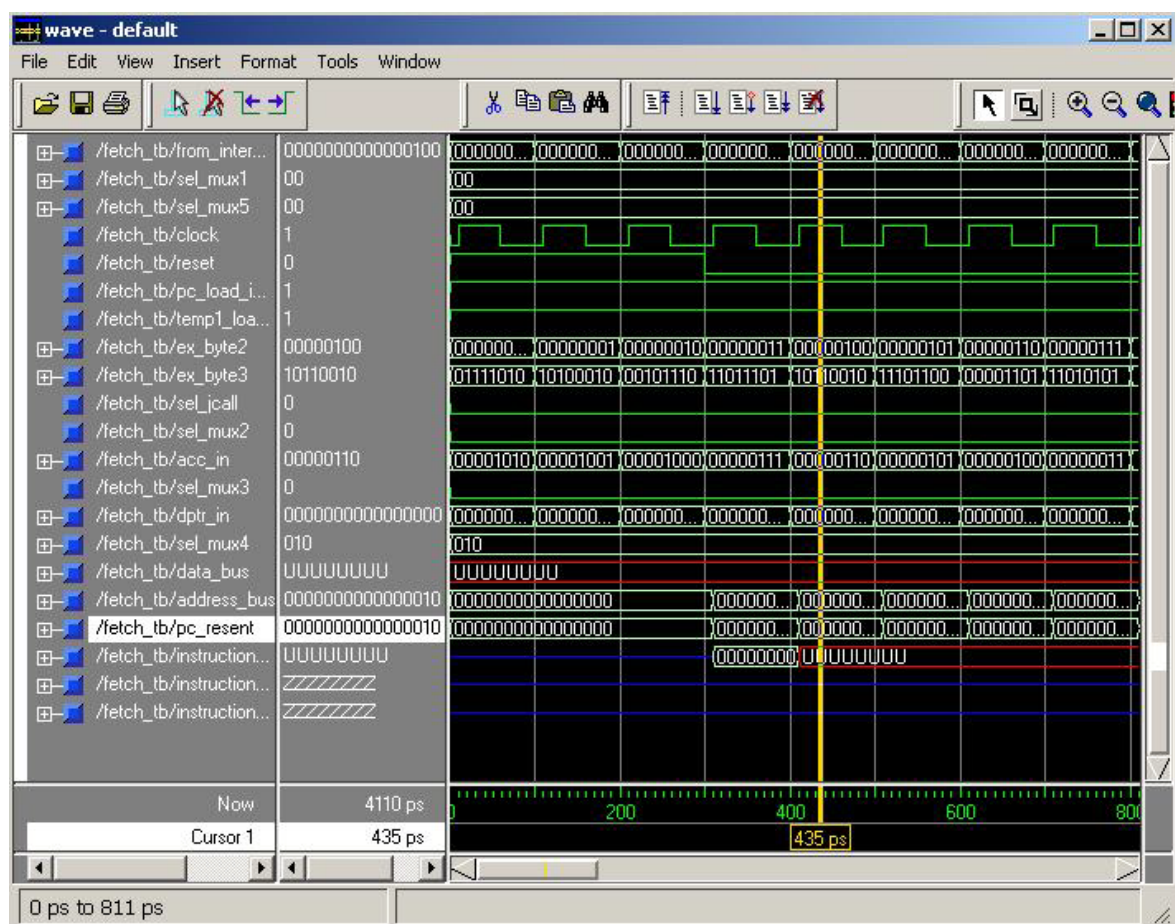


รูปที่ 6.1 การทดสอบการทำงานของ ALU

6.1.2 การทดสอบในระดับไปป์ไลน์สเตจ

หลังจากนำเอาวงจรย่อยมาประกอบเข้าเป็นสเตจการทำงานแล้ว ต้องมีการทดสอบการทำงานของไปป์ไลน์สเตจด้วยเพื่อดูการทำงานของสเตจนั้น ๆ และดูผลจากการทำงานเพื่อส่งต่อไปทำงานยังสเตจถัดไปว่ามีความถูกต้องหรือไม่ การทดสอบมีในหลายๆส่วน หลักๆได้แก่

- ทดสอบการทำงานเมื่อมีการทำคำสั่งที่มีการขึ้นต่อกันของข้อมูลว่ายังทำงานกับข้อมูลที่ถูกต้อง
- ทดสอบการอ้างถึงโอเปอร์เรนด์ต่างๆ ว่าอ้างได้ถูกต้องสมบูรณ์
- ทดสอบการทำคำสั่งในกลุ่มกระโดด ว่ามีการทำงานที่เป็นปกติหรือไม่ เมื่อมีการกระโดดสามารถไปทำงานยังตำแหน่งที่ต้องการได้ แลหากไม่มีการกระโดด สามารถทำงานในตำแหน่งเดิมได้เป็นปกติ
- ทดสอบการเข้าถึงหน่วยความจำทั้งการอ่านข้อมูล และการเขียนข้อมูลได้อย่างถูกต้อง



รูปที่ 6.2 การทดสอบการทำงานของสเตจการอ่านคำสั่ง

จากรูปที่ 6.2 เป็นการทดสอบการทำงานของสแต็กการอ่านคำสั่ง โดยการตรวจสอบว่ามีการอ่านคำสั่งขึ้นมาถูกต้องหรือไม่ ครอบคลุมจำนวนไบต์ของคำสั่งหรือไม่ สามารถจัดการค่าพีซีที่นับการทำคำสั่งได้ถูกต้องหรือไม่ เป็นต้น

6.1.3 การทดสอบในระดับวงจรรวม

เมื่อทำการรวมการทำงานของสแต็กการทำงานทั้งหมดเข้าเป็นซีพียูแล้ว จึงทำการทดสอบการทำงานรวมทั้งหมดอีกครั้ง การทดสอบในระดับนี้จะดูถึงการทำงานของซีพียูทั้งหมดในการทำคำสั่งต่างๆ ว่ามีผลการการทำงานที่ถูกต้อง มีการส่งข้อมูลออกมายังพอร์ตที่กำหนดไว้ถูกต้องตามโปรแกรม เป็นต้น

การทดสอบการทำงานของวงจรนี้เรียกว่าเป็น การทดสอบในระดับฟังก์ชันการทำงาน หรือ Functional Simulation โดยหลังจากการทดสอบในระดับฟังก์ชันการทำงานของวงจรรวมแล้ว เมื่อทำการสังเคราะห์วงจรและทำการสร้างไฟล์การทำงานของวงจรเพื่อทำการบันทึกลงเฟิร์มแวร์แล้วก่อนการบันทึก จะมีการทดสอบอีกครั้งโดยการจำลองการทำงานเช่นกันเรียกว่าการจำลองการทำงานของวงจรในระดับ Post Layout Simulation

6.2 การทดสอบการทำงานของคำสั่งต่างๆ

คำสั่งของไมโครคอนโทรลเลอร์ต้นแบบมีทั้งสิ้น 111 คำสั่ง แบ่งออกเป็น 6 ประเภท ดังนี้

1. คำสั่ง 1 byte – 1 cycle จำนวน 37 คำสั่ง
2. คำสั่ง 1 byte – 2 cycles จำนวน 10 คำสั่ง
3. คำสั่ง 1 byte – 4 cycles จำนวน 2 คำสั่ง
4. คำสั่ง 2 byte – 1 cycle จำนวน 27 คำสั่ง
5. คำสั่ง 2 byte – 2 cycles จำนวน 19 คำสั่ง
6. คำสั่ง 3 byte – 2 cycles จำนวน 16 คำสั่ง

คำสั่งต่างๆของไมโครคอนโทรลเลอร์ที่ออกแบบได้มีทั้งสิ้น 62 คำสั่ง แบ่งได้ตามจำนวน cycle ในการทำงาน โดย คำสั่งที่ทำงานได้ภายใน 1 cycle มี 30 คำสั่ง ดังนี้

MOV A,Rn	MOV A,@Ri	MOV Rn,A	MOV @Ri,A
ORL A,Rn	ORL A,@Ri	ANL A,Rn	ANL A,@Ri
XRL A,Rn	XRL A,@Ri	RL A	RLC A
RR A	RRC A	SWAP A	CPL A
CLR A	DA A	ADD A,Rn	ADD A,@Ri
ADDC A,Rn	ADDC A,@Ri	SUBB A,Rn	SUBB A,@Ri
XCH A,Rn	XCH A,@Ri	XCHD A,@Ri	NOP
INC A	DEC A		

คำสั่งที่ทำงานได้ภายใน 2 cycles มี 25 คำสั่ง ดังนี้

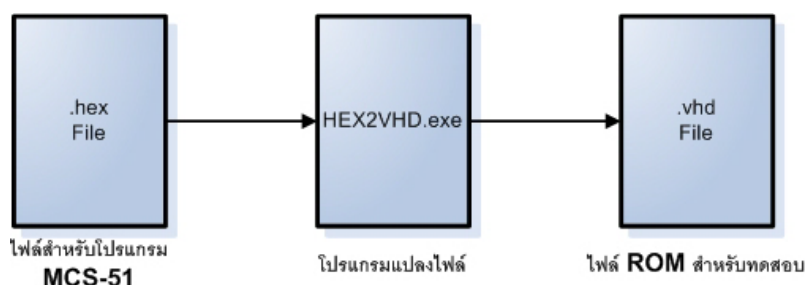
MOV A,direct	MOV A,#data	MOV Rn,direct	MOV Rn,#data
MOV direct,A	MOV direct,Rn	MOV direct,@Ri	MOV @Ri,direct
MOV @Ri,#data	ORL A,direct	ORL A,#data	ORL direct,A
ANL A,direct	ANL A,#data	ANL direct,A	XRL A,direct
XRL A,#data	XRL direct,A	ADD A,direct	ADD A,#data
ADDC A,direct	ADDC A,#data	SUBB A,direct	SUBB A,#data
AJMP ADDR11			

คำสั่งที่ทำงานได้ภายใน 3 cycles มี 7 คำสั่ง ดังนี้

MOV direct,direct	MOV direct,#data	ORL direct,#data	ANL direct,#data
XRL direct,#data	LJMP ADDR16	MOV DPTR,#data16	

6.2.1 วิธีการทดสอบการทำคำสั่ง

ในการทดสอบการทำคำสั่ง จะต้องทำการเขียนรอมภายในไมโครคอนโทรลเลอร์ที่ใช้บรรจุคำสั่งขึ้นมา ด้วยภาษาแอสเซมบลีและใช้การทดสอบในระดับวงจรรวม โดยรอมนี้จะได้จากการนำเอาโปรแกรมที่เขียนขึ้นด้วย ภาษาต่างๆ เพื่อให้ง่ายควรจะใช้ภาษาแอสเซมบลีและคอมไพล์เป็นไฟล์ .hex ของอินเทลซึ่งสามารถใช้งานกับ ไมโครคอนโทรลเลอร์ต้นแบบได้จริง มาทำการแปลงด้วยโปรแกรมแปลงไฟล์ซึ่งเขียนด้วยภาษาปาสคาล ให้อยู่ใน รูปของไฟล์รอมที่ของภาษาแอสเซมบลีแล้วนำไปเป็นรอมภายในตามต้องการ ดังรูปที่ 6.3 เมื่อทำการทดสอบการ ทำงานตัวไมโครคอนโทรลเลอร์ก็จะทำการอ่านคำสั่งที่อยู่ในรอมขึ้นมาทีละคำสั่ง และทำงานให้ผลลัพธ์ออกมาให้ ตรวจสอบ โดยดูที่ ค่าของรีจิสเตอร์ ค่าของหน่วยความจำตำแหน่งในตำแหน่งต่างๆ ที่ถูกอ่าน-เขียน ในการทดสอบ จะแบ่งการทดสอบคำสั่งต่างๆ โดยเขียนคำสั่งในหมวดหมู่เดียวกันไว้ในโปรแกรมเดียวกันเพื่อให้สามารถดูผลการ ทำงานได้ง่าย รอมภายในสำหรับทดสอบจึงแบ่งเป็นหลายตัว เมื่อทำการทดสอบคำสั่งชุดหนึ่งเสร็จสิ้นและมีการ ทำงานที่ถูกต้อง ก็จะทำการเปลี่ยนรอมภายในของชุดคำสั่งอื่นๆ เข้ามาใหม่ โดยรอมภายในแต่ละตัวมีคำสั่งที่ ทดสอบประมาณ 3-5 คำสั่ง การทดสอบคำสั่งทั้ง 62 คำสั่งในที่นี่จะใช้รอมจำนวน 15 ตัว



รูปที่ 6.3 ขั้นตอนการสร้างรอมภายในที่ใช้ทดสอบ

6.2.2 การทดสอบการทำคำสั่งของรอมภายในแต่ละตัว

ในการทำการทดสอบ จะดูผลลัพธ์ที่เกิดขึ้นกับรีจิสเตอร์แอดคิวมูลเตอร์เป็นหลัก และค่าของแฟลกต่างๆ ตลอดจนค่าของตำแหน่งหน่วยความจำที่มีการเขียนและอ่านในแต่ละขั้นตอนของการทำงาน จึงมีการกำหนดสัญญาณส่งออกมาเป็นเอาต์พุต แสดงค่าดังนี้ คำสั่งไบต์ที่ 1, 2 และ 3 ค่าแอดคิวมูลเตอร์ ค่าแฟลก ตำแหน่งหน่วยความจำที่ถูกอ่านและข้อมูลที่อ่านได้ ตำแหน่งหน่วยความจำที่ถูกเขียนและข้อมูลที่เขียน โดยสัญญาณดังกล่าวจะไม่ถูกนำมาใช้เป็นเอาต์พุตของวงจรเมื่อทำการบันทึกลงอุปกรณ์จริง

ในหัวข้อถัดไปจะเป็นการทดสอบการทำคำสั่งต่างๆ ในรอมแต่ละตัว โดยการทดสอบจะต้องมีการสังเกตค่าที่ของ Waveform ที่เกิดขึ้น เพื่อให้สามารถสังเกตได้จึงขออธิบายการอ่าน Waveform ที่เกิดขึ้นดังนี้ ในการทำงานในจังหวะหนึ่งของไมโครคอนโทรลเลอร์จะใช้สัญญาณนาฬิกา 1 ลูกโดยเมื่อสัญญาณนาฬิกาอยู่ที่ขอบขาขึ้นแล้วมีคำสั่งไบต์ที่ 1 เกิดขึ้น (อาจจะมีไบต์ที่ 2 หรือ 3 เกิดขึ้นด้วย แล้วแต่คำสั่ง)จะถือว่าคำสั่งนั้นได้ทำงานในสเตจการ Fetch เสร็จสิ้นแล้ว ให้เริ่มสังเกตค่าต่างๆ หลังจากสเตจการ Fetch ว่าถูกต้องหรือไม่ เช่น ค่าของตำแหน่งหน่วยความจำในไบต์ที่ 2 หรือค่าคงที่ของคำสั่งป้อนเข้ามาในไบต์ที่ 3 เป็นต้น เมื่อเลื่อนไปยังสัญญาณนาฬิกาถูกกดไปให้นับเป็นสเตจการ Decode ที่แสดงนี้ให้สังเกตการอ่านค่าจากหน่วยความจำ การกำหนดค่าของโอปอเรนด์ที่จะเข้าไปยังสเตจถัดไปว่าสามารถทำได้ถูกต้องหรือไม่ จากนั้นเข้าสู่สัญญาณนาฬิกาถูกกดไป ในลูกนี้ให้ถือว่ามีการทำงานของสเตจ Execute ค่าของคำสั่งนั้นๆแล้ว ให้สังเกตค่าที่ได้จากการทำคำสั่งว่าผลลัพธ์ที่ออกมานั้นมีความถูกต้องหรือไม่ จากนั้นเมื่อถึงสัญญาณนาฬิกาถูกกดไปจะเป็นการทำงานของสเตจ Writeback ที่แสดงนี้ให้ทำการสังเกตผลของข้อมูลที่เขียนลงไปยังรีจิสเตอร์หรือหน่วยความจำในตำแหน่งต่างๆ ว่ามีความถูกต้องตรงตามโปรแกรมที่ได้ทำการเขียนไว้หรือไม่ ถ้าถูกต้องถือว่าคำสั่งดังกล่าวได้ทำงานได้อย่างถูกต้องแล้ว หากต้องการดูคำสั่งในการทำงานใดๆ จึงควรเริ่มการสังเกตจากสเตจ Fetch ก่อน โดยการหาดำแหน่งที่มีการ Fetch คำสั่งขึ้นมาครบถ้วน ตรงตามคำสั่งของไมโครคอนโทรลเลอร์ต้นแบบ แล้วจึงไล่สเตจการทำงานและดูผลการทำงานแต่ละสเตจไปเรื่อยๆ ตามสัญญาณนาฬิกา ผลการทดสอบการทำงานของรอมภายในต่างๆ มีดังนี้

6.2.2.1 รอมภายในตัวที่ 1

โปรแกรมของรอมภายในตัวที่ 1 ประกอบด้วยคำสั่งต่างๆ ดังรูปที่ 6.4 โดยการทดสอบการทำงานของคำสั่ง NOP และทำการกระโดดแบบไม่มีเงื่อนไขด้วยคำสั่ง LJMP เพื่อให้โปรแกรมจำลองการทำงานวนลูป

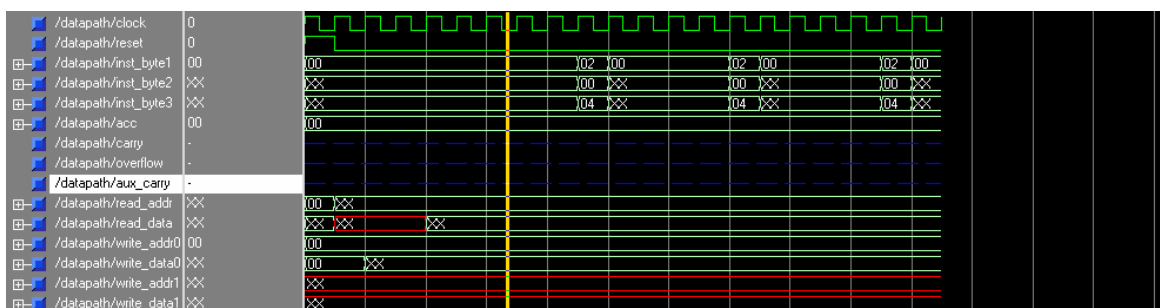
```

| ORG      0
| :Test NOP Operation
| NOP
| NOP
| NOP
| NOP
| -----
| MAIN:
|   LJMP   MAIN
|   NOP
|   NOP
|   NOP
|   NOP
|   NOP

```

รูปที่ 6.4 คำสั่งของรอมภายในตัวที่ 1

ในการตรวจสอบให้สังเกตค่าของคำสั่งที่อ่านออกมาจำนวน 3 ไบต์จะไม่มีค่าของคำสั่งเนื่องจากเป็นคำสั่ง NOP และไม่มีการประมวลผลใดๆ เมื่อถึงคำสั่ง LJMP จะพบว่ามีการกระโดดอย่างไม่มีเงื่อนไขขึ้น ดังรูปที่ 6.5



รูปที่ 6.5 ผลการจำลองการทำงานของรอมภายในตัวที่ 1

6.2.2.2 รอมภายในตัวที่ 2

โปรแกรมของรอมตัวที่ 2 แสดงดังรูปที่ 6.6 เป็นการทำงานของคำสั่ง MOV ที่เกี่ยวข้องกับการย้ายค่าไปเก็บไว้ในแอดคิวมูเลเตอร์ซึ่งจะมีอยู่ 4 รูปแบบด้วยกัน คือ การย้ายค่าจาก รีจิสเตอร์ใช้งานทั่วไป, หน่วยความจำที่อ้างถึงโดยตรง, หน่วยความจำที่อ้างถึงโดยอ้อม และค่าคงที่ จากนั้นทำการวนลูปเช่นเดียวกับรอมตัวที่ 1 แต่อาศัยการทำงานของคำสั่ง AJMP แทน

```

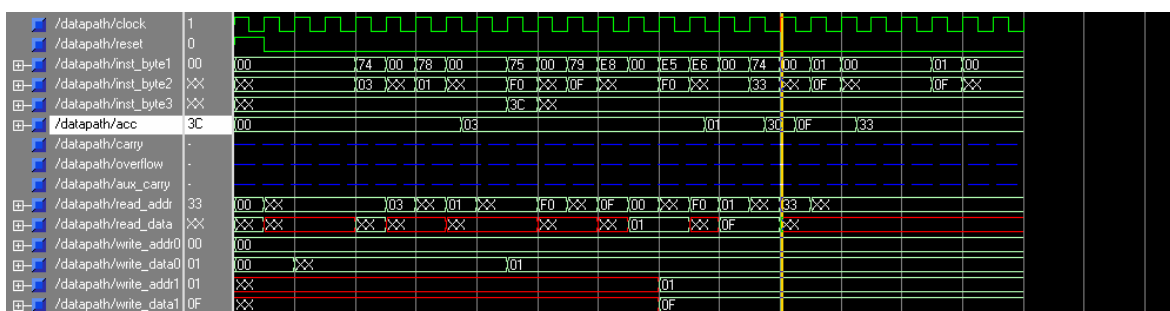
ORG      0
;Basic Data for Test
MOV      A, #00000011B           ;mov    a,data
MOV      R0, #00000001B          ;mov    Rn,data
MOV      0F0H, #00111100B        ;mov    direct,data
MOV      R1, #00001111B
;Test MOV Operation - MOV A,???
MOV      A, R0                   ;mov    a,Rn
MOV      A, 0F0H                 ;mov    a,direct
MOV      A, @R0                  ;mov    a,@Ri
MOV      A, #00110011B           ;mov    a,data

MAIN:
AJMP     MAIN
NOP
NOP
NOP
NOP
NOP

```

รูปที่ 6.6 คำสั่งของรอมภายในตัวที่ 2

การทำงานส่วนแรกเป็นการกำหนดค่าต่างๆให้กับโปรแกรม จากนั้นจะทำการทดสอบการย้ายค่าเข้าสู่แอดคิวมูเลเตอร์ พบว่าสามารถทำงานได้อย่างถูกต้องในทุกรูปแบบของคำสั่ง โดยสังเกตค่าของแอดคิวมูเลเตอร์ที่เปลี่ยนแปลงไปในทุกๆ การทำคำสั่งเทียบกับค่าในโปรแกรมที่ป้อนเข้ามา ดังรูปที่ 6.7



รูปที่ 6.7 ผลการจำลองการทำงานของรอมภายในตัวที่ 2

6.2.2.3 รอมภายในตัวที่ 3

โปรแกรมของรอมตัวที่ 2 แสดงดังรูปที่ 6.8 เป็นการทำงานของคำสั่ง MOV ที่เกี่ยวข้องกับการย้ายค่าไปเก็บไว้ในรีจิสเตอร์ใช้งานทั่วไป ซึ่งจะมีอยู่ 3 รูปแบบด้วยกัน คือ การย้ายค่าจาก แอคคิวมูเลเตอร์, หน่วยความจำที่อ้างถึงโดยตรง และค่าคงที่ จากนั้นทำการวนลูปเช่นเดียวกับรอมตัวที่ 2 โดยอาศัยการทำงานของคำสั่ง AJMP

```

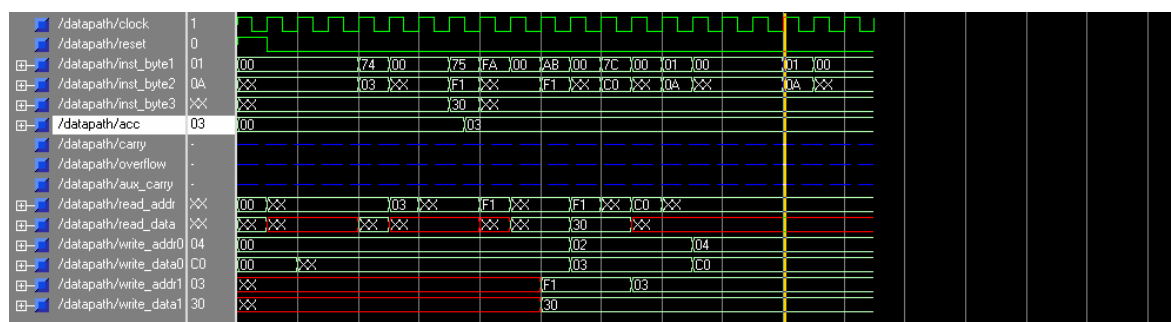
ORG      0
;Basic Data for Test
MOV      A, #00000011B           ;mov    a,data
MOV      0F1H, #00110000B        ;mov    .....
;Test MOV Operation - MOV Rn,???
MOV      R2, A                   ;mov    Rn,A
MOV      R3, 0F1H                 ;mov    Rn,direct
MOV      R4, #11000000B          ;mov    Rn,data

MAIN:
AJMP     MAIN
NOP
NOP
NOP
NOP
NOP

```

รูปที่ 6.8 คำสั่งของรอมภายในตัวที่ 3

การทำงานส่วนแรกเป็นการกำหนดค่าต่างๆให้กับโปรแกรม จากนั้นจะทำการทดสอบการย้ายค่าเข้าสู่รีจิสเตอร์ใช้งานทั่วไป พบว่าสามารถทำงานได้อย่างถูกต้องในทุกรูปแบบของคำสั่ง โดยสังเกตค่าของตำแหน่งแอคเครสของรีจิสเตอร์ใช้งานทั่วไปที่ถูกเขียน และข้อมูลที่เขียนซึ่งที่เปลี่ยนแปลงไปในทุกๆ การทำคำสั่งเทียบกับค่าในโปรแกรมที่ป้อนเข้ามา ดังรูปที่ 6.9



รูปที่ 6.9 ผลการจำลองการทำงานของรอมภายในตัวที่ 3

6.2.2.4 รอมภายในตัวที่ 4

โปรแกรมของรอมตัวที่ 4 แสดงดังรูปที่ 6.10 เป็นการทำงานของคำสั่ง MOV ที่เกี่ยวข้องกับการย้ายค่าไปเก็บไว้ในหน่วยความจำโดยตรงแบบไบต์ ซึ่งจะมีอยู่ 5 รูปแบบด้วยกัน คือ การย้ายค่าจาก แอคคิวมูเลเตอร์, หน่วยความจำที่อ้างถึงโดยอ้อม และค่าคงที่ จากนั้นทำการวนลูปเช่นเดียวกับรอมตัวที่ 2 โดยอาศัยการทำงานของคำสั่ง AJMP

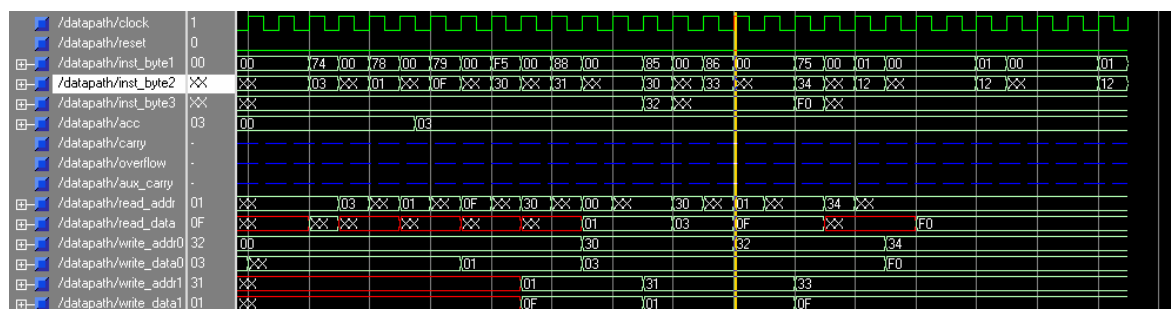
```

ORG      0
;Basic Data for Test
MOV      A, #00000011B           ;mov    a,data
MOV      R0, #00000001B          ;mov    Rn,data
MOV      R1, #00001111B          ;mov    Rn,data
;Test MOV Operation - MOV Direct, ???
MOV      30H, A                  ;mov    direct,A
MOV      31H, R0                  ;mov    direct,Rn
MOV      32H, 30H                 ;mov    direct,direct
MOV      33H, @R0                 ;mov    direct,@Ri
MOV      34H, #11110000B         ;mov    direct,#data
MAIN:
AJMP     MAIN
NOP
NOP
NOP
NOP
NOP

```

รูปที่ 6.10 คำสั่งของรอมภายในตัวที่ 4

การทำงานส่วนแรกเป็นการกำหนดค่าต่างๆ ให้กับโปรแกรม จากนั้นจะทำการทดสอบการย้ายค่าเข้าสู่หน่วยความจำพบว่าสามารถทำงานได้อย่างถูกต้องในทุกรูปแบบของคำสั่ง โดยสังเกตค่าของตำแหน่งแอดเดรสของหน่วยความจำที่ถูกเขียน และข้อมูลที่เขียนซึ่งที่เปลี่ยนแปลงไปในทุกๆ การทำคำสั่งเทียบกับค่าในโปรแกรมที่ป้อนเข้ามา ดังรูปที่ 6.11



รูปที่ 6.11 ผลการจำลองการทำงานของรอมภายในตัวที่ 4

6.2.2.5 รอมภายในตัวที่ 5

โปรแกรมของรอมตัวที่ 5 แสดงดังรูปที่ 6.12 เป็นการทำงานของคำสั่ง MOV ที่เกี่ยวข้องกับการย้ายค่าไปเก็บไว้ในหน่วยความจำโดยอ้อม ซึ่งจะมียู่ 3 รูปแบบด้วยกัน คือ การย้ายค่าจาก แอคคิวมูเลเตอร์, หน่วยความจำที่อ้างถึงโดยตรง และค่าคงที่ จากนั้นทำการวนลูปเช่นเดียวกับรอมตัวที่ 2 โดยอาศัยการทำงานของคำสั่ง AJMP

```

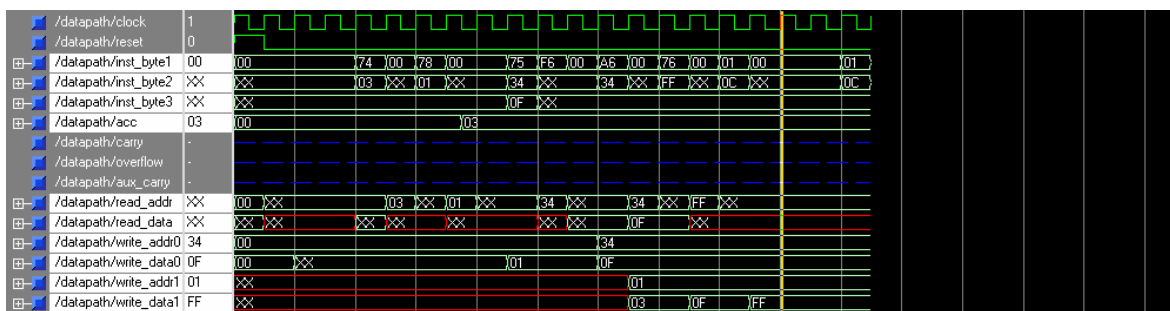
ORG      0
;Basic Data for Test
MOV      A, #00000011B           ;mov    a,data
MOV      R0, #00000001B          ;mov    Rn,data
MOV      34H, #00001111B         ;mov    direct,data
;Test MOV Operation - MOV Direct, ???
MOV      @R0, A                  ;mov    @Ri, A
MOV      @R0, 34H                ;mov    @Ri, direct
MOV      @R0, #11111111B         ;mov    @Ri, data

MAIN:
AJMP     MAIN
NOP
NOP
NOP
NOP
NOP
NOP

```

รูปที่ 6.12 คำสั่งของรอมภายในตัวที่ 5

การทำงานส่วนแรกเป็นการกำหนดค่าต่างๆให้กับโปรแกรม จากนั้นจะทำการทดสอบการย้ายค่าเข้าสู่หน่วยความจำที่อ้างถึงโดยอ้อมพบว่าสามารถทำงานได้อย่างถูกต้องในทุกรูปแบบของคำสั่ง โดยสังเกตค่าของตำแหน่งแอดเดรสของหน่วยความจำที่ถูกเขียน และข้อมูลที่เขียนซึ่งที่เปลี่ยนแปลงไปในทุกๆ การทำคำสั่งเทียบกับค่าในโปรแกรมที่ป้อนเข้ามา ดังรูปที่ 6.13



รูปที่ 6.13 ผลการจำลองการทำงานของรอมภายในตัวที่ 5

6.2.2.6 รอมภายในตัวที่ 6

โปรแกรมของรอมตัวที่ 6 แสดงดังรูปที่ 6.14 เป็นการทำงานของคำสั่ง ORL ซึ่งแบ่งเป็น 2 ประเภท ประเภทแรกคือการทำคำสั่งระหว่างแอดคิวมูลเตอร์กับ รีจิสเตอร์ใช้งานทั่วไป หน่วยความจำที่อ้างถึงโดยตรง, หน่วยความจำที่อ้างถึงโดยอ้อม และค่าคงที่ ประเภทที่สองคือการทำคำสั่งระหว่างหน่วยความจำที่อ้างถึงโดยตรงกับแอดคิวมูลเตอร์ และค่าคงที่ จากนั้นทำการวนลูปเช่นเดียวกับรอมตัวที่ 2 โดยอาศัยการทำงานของคำสั่ง AJMP

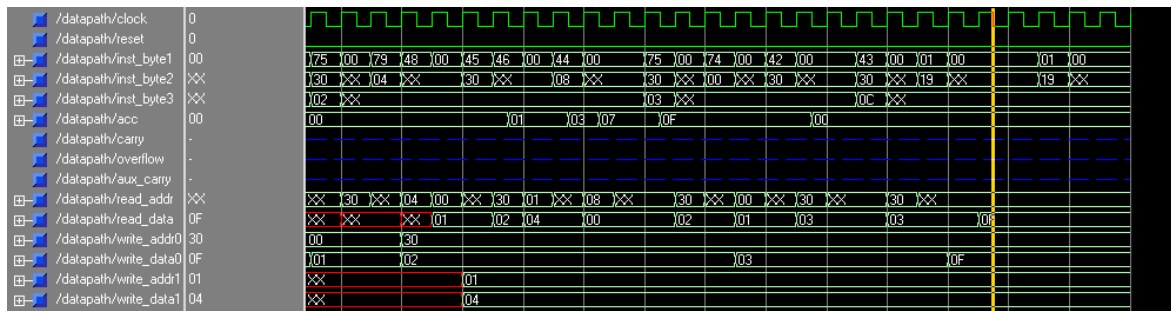
```

ORG      0
;Basic Data for Test
MOV      A,#00000000B
MOV      R0,#00000001B
MOV      30H,#00000010B
MOV      R1,#00000100B
;Test ORL Operation - ORL A,???
ORL      A,R0
ORL      A,30H
ORL      A,@R0
ORL      A,#00001000B
;Basic Data for Test
MOV      30H,#00000011B
MOV      A,#00000000B
;Test ORL Operation - ORL direct,???
ORL      30H,A
ORL      30H,#00001100B
MAIN:
AJMP     MAIN
NOP
NOP
NOP
NOP
NOP

```

รูปที่ 6.14 คำสั่งของรอมภายในตัวที่ 6

การทำงานส่วนแรกเป็นการกำหนดค่าต่างๆให้กับโปรแกรม จากนั้นจะทำการทดสอบการทำคำสั่ง ORL ระหว่างแอดคิวมูลเตอร์กับค่าต่างๆ พบว่าสามารถทำงานได้อย่างถูกต้องในทุกรูปแบบของคำสั่ง โดยสังเกตค่าของแอดคิวมูลเตอร์ที่ถูกเขียน ส่วนที่สองเป็นการกำหนดค่าและทดสอบการทำคำสั่ง ORL ระหว่างหน่วยความจำโดยตรงกับค่าต่างๆ พบว่าสามารถทำงานได้อย่างถูกต้องโดยสังเกตค่าของตำแหน่งแอดเดรสของหน่วยความจำที่ถูกเขียน และข้อมูลที่เขียนซึ่งที่เปลี่ยนแปลงไปในทุกๆ ดังรูปที่ 6.15



รูปที่ 6.15 ผลการจำลองการทำงานของรอมภายในตัวที่ 6

6.2.2.7 รอมภายในตัวที่ 7

โปรแกรมของรอมตัวที่ 7 แสดงดังรูปที่ 6.16 เป็นการทำงานของคำสั่ง ANL ซึ่งแบ่งเป็น 2 ประเภท ประเภทแรกคือการทำคำสั่งระหว่างแอดคิวมูลเตอร์กับ รีจิสเตอร์ใช้งานทั่วไป หน่วยความจำที่อ้างถึงโดยตรง, หน่วยความจำที่อ้างถึงโดยอ้อม และค่าคงที่ ประเภทที่สองคือการทำคำสั่งระหว่างหน่วยความจำที่อ้างถึงโดยตรงกับแอดคิวมูลเตอร์ และค่าคงที่ จากนั้นทำการวนลูปเช่นเดียวกับรอมตัวที่ 2 โดยอาศัยการทำงานของคำสั่ง AJMP

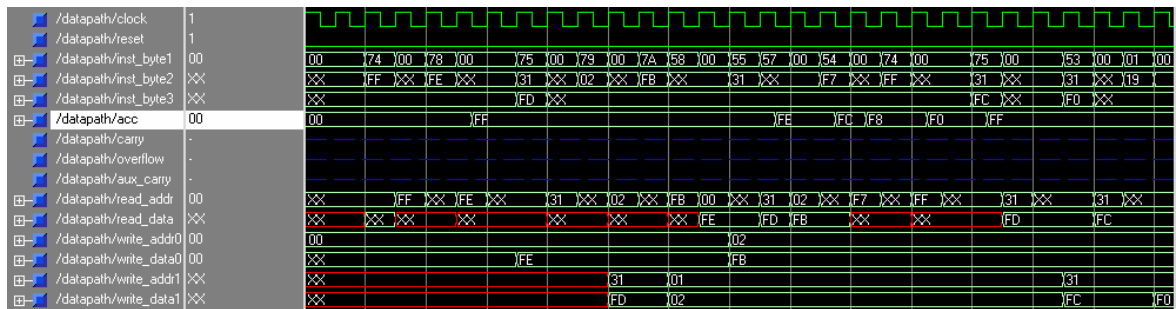
```

ORG      0
;Basic Data for Test
MOV      A,#11111111B
MOV      R0,#11111110B
MOV      31H,#11111101B
MOV      R1,#00000010B
MOV      R2,#11111011B
;Test ANL Operation - ANL A,???
ANL      A,R0
ANL      A,31H
ANL      A,@R1
ANL      A,#11110111B
;Basic Data for Test
MOV      31H,#11111100B
MOV      A,#11111111B
;Test ANL Operation - ANL direct,???
ANL      31H,A
ANL      31H,#11110000B
MAIN:
AJMP     MAIN
NOP
NOP
NOP
NOP
NOP

```

รูปที่ 6.16 คำสั่งของรอมภายในตัวที่ 7

การทำงานส่วนแรกเป็นการกำหนดค่าต่างๆ ให้กับโปรแกรม จากนั้นจะทำการทดสอบการทำคำสั่ง ANL ระหว่างแอดคิวมูลเตอร์กับค่าต่างๆ พบว่าสามารถทำงานได้อย่างถูกต้องในทุกรูปแบบของข้อมูล โดยสังเกตค่าของแอดคิวมูลเตอร์ที่ถูกเขียน ส่วนที่สองเป็นการกำหนดค่าและทดสอบการทำคำสั่ง ANL ระหว่างหน่วยความจำ โดยตรงกับค่าต่างๆ พบว่าสามารถทำงานได้อย่างถูกต้องโดยสังเกตค่าของตำแหน่งแอดเดรสของหน่วยความจำที่ถูกเขียน และข้อมูลที่เขียนซึ่งที่เปลี่ยนแปลงไปในทุกๆ ดังรูปที่ 6.17



รูปที่ 6.17 ผลการจำลองการทำงานของรอมภายในตัวที่ 7

6.2.2.8 รอมภายในตัวที่ 8

โปรแกรมของรอมตัวที่ 8 แสดงดังรูปที่ 6.18 เป็นการทำงานของคำสั่ง XRL ซึ่งแบ่งเป็น 2 ประเภท ประเภทแรกคือการทำคำสั่งระหว่างแอดคิวมูลเตอร์กับ รีจิสเตอร์ใช้งานทั่วไป หน่วยความจำที่อ้างถึงโดยตรง, หน่วยความจำที่อ้างถึงโดยอ้อม และค่าคงที่ ประเภทที่สองคือการทำคำสั่งระหว่างหน่วยความจำที่อ้างถึงโดยตรงกับแอดคิวมูลเตอร์ และค่าคงที่ จากนั้นทำการวนลูปเช่นเดียวกับรอมตัวที่ 2 โดยอาศัยการทำงานของคำสั่ง AJMP

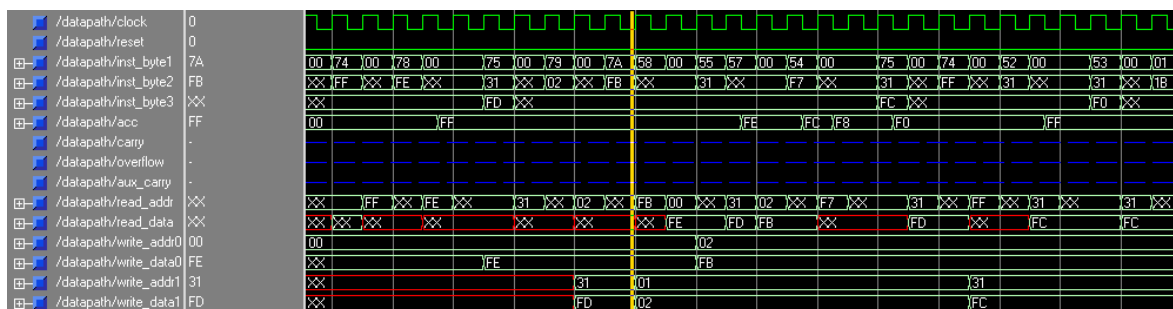
การทำงานส่วนแรกเป็นการกำหนดค่าต่างๆ ให้กับโปรแกรม จากนั้นจะทำการทดสอบการทำคำสั่ง XRL ระหว่างแอดคิวมูลเตอร์กับค่าต่างๆ พบว่าสามารถทำงานได้อย่างถูกต้องในทุกรูปแบบของข้อมูล โดยสังเกตค่าของแอดคิวมูลเตอร์ที่ถูกเขียน ส่วนที่สองเป็นการกำหนดค่าและทดสอบการทำคำสั่ง XRL ระหว่างหน่วยความจำ โดยตรงกับค่าต่างๆ พบว่าสามารถทำงานได้อย่างถูกต้องโดยสังเกตค่าของตำแหน่งแอดเดรสของหน่วยความจำที่ถูกเขียน และข้อมูลที่เขียนซึ่งที่เปลี่ยนแปลงไปในทุกๆ ดังรูปที่ 6.19

```

ORG      Undo
;Basic Data for Test
MOV      A,#11111111B
MOV      R0,#11111110B
MOV      32H,#11111101B
MOV      R1,#00000010B
MOV      R2,#11111011B
;Test XRL Operation - XRL A,???
XRL      A,R0
XRL      A,32H
XRL      A,@R1
XRL      A,#11110111B
;Basic Data for Test
MOV      31H,#11111100B
MOV      A,#11111111B
;Test XRL Operation - XRL direct,???
XRL      31H,A
XRL      31H,#11110000B
MAIN:
AJMP     MAIN
NOP
NOP
NOP
NOP
NOP

```

รูปที่ 6.18 คำสั่งของรอมภายในตัวที่ 8



รูปที่ 6.19 ผลการจำลองการทำงานของรอมภายในตัวที่ 8

6.2.2.9 รอมภายในตัวที่ 9

โปรแกรมของรอมตัวที่ 9 แสดงดังรูปที่ 6.20 เป็นการทำงานของคำสั่ง RL RLC RR RRC ซึ่งทำงานกับแอดคิวมูลเตอร์ นอกจากนี้ยังมีคำสั่ง SWAP CPL และ CLR ซึ่งทำงานกับแอดคิวมูลเตอร์เช่นกัน

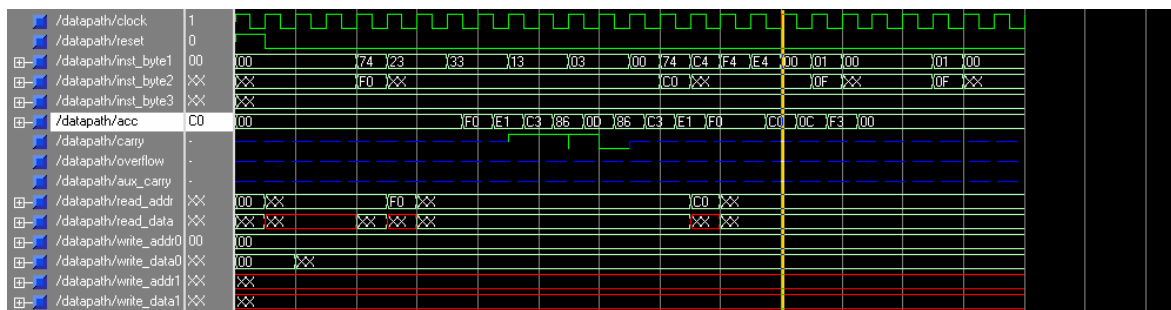
การทำงานส่วนแรกเป็นการกำหนดค่าต่างๆให้กับโปรแกรม จากนั้นจะทำการทดสอบการทำคำสั่ง RL RLC RRC RR ตามลำดับ การตรวจสอบทำได้โดยการสังเกตการเปลี่ยนแปลงค่าของแอดคิวมูลเตอร์ และค่าของแฟล็กทกที่เกิดขึ้น พบว่าสามารถทำงานได้อย่างถูกต้อง ในส่วนที่สองก็เช่นกัน สามารถสังเกตการทำคำสั่งโดยดูผลจากค่าของแอดคิวมูลเตอร์ที่เปลี่ยนแปลงไป ดังรูปที่ 6.21

```

    ORG      0
;Basic Data for Test
    MOV     A,#111110000B
;Test RL,RR,RLC,RRC Operation
    RL      A
    RL      A
    RLC     A
    RLC     A
    RRC     A
    RRC     A
    RR      A
    RR      A
;Basic Data for Test
    MOV     A,#110000000B
;Test SWAP Operation
    SWAP    A
;Test CPL Operation
    CPL     A
;Test CLR Operation
    CLR     A
MAIN:
    AJMP    MAIN
    NOP
    NOP
    NOP
    NOP
    NOP

```

รูปที่ 6.20 คำสั่งของรอมภายในตัวที่ 9



รูปที่ 6.21 ผลการจำลองการทำงานของรอมภายในตัวที่ 9

6.2.2.10 รอมภายในตัวที่ 10

โปรแกรมของรอมตัวที่ 10 แสดงดังรูปที่ 6.22 เป็นการทำงานของคำสั่ง ADD ระหว่างแอดคิวมูลเตอร์กับ จาก รีจิสเตอร์ใช้งานทั่วไป, หน่วยความจำที่อ้างถึงโดยตรง, หน่วยความจำที่อ้างถึงโดยอ้อม และค่าคงที่

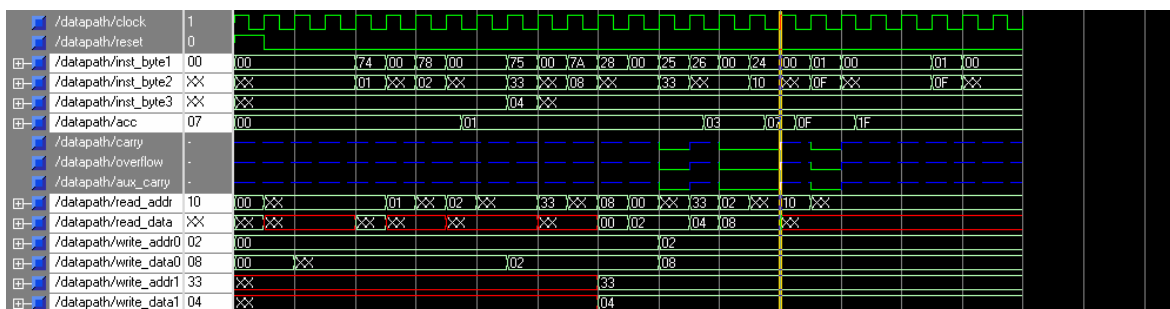
```

    ORG      0
;Basic Data for Test
    MOV     A, #00000001B
    MOV     R0, #00000010B
    MOV     33H, #00000100B
    MOV     R2, #00001000B
;Test ADD Operation - ADD A, ???
    ADD     A, R0
    ADD     A, 33H
    ADD     A, @R0
    ADD     A, #00010000B
MAIN:
    AJMP    MAIN
    NOP
    NOP
    NOP
    NOP
    NOP

```

รูปที่ 6.22 คำสั่งของรอมภายในตัวที่ 10

การทำงานส่วนแรกเป็นการกำหนดค่าต่างๆให้กับโปรแกรม จากนั้นจะทำการทดสอบการทำคำสั่ง ADD ระหว่างแอดคิวมูลเตอร์กับค่าต่างๆ พบว่าสามารถทำงานได้อย่างถูกต้องในทุกรูปแบบของข้อมูล โดยสังเกตค่าของแอดคิวมูลเตอร์ที่ถูกเขียน ดังรูปที่ 6.23



รูปที่ 6.23 ผลการจำลองการทำงานของรอมภายในตัวที่ 10

6.2.2.11 รอมภายในตัวที่ 11

โปรแกรมของรอมตัวที่ 11 แสดงดังรูปที่ 6.24 เป็นการทำงานของคำสั่ง ADDC ระหว่างแอดคิวิตูเลเตอร์กับ จา กริ จิสเตอร์ใช้งานทั่วไป, หน่วยความจำที่อ้างถึงโดยตรง, หน่วยความจำที่อ้างถึงโดยอ้อม และค่าคงที่

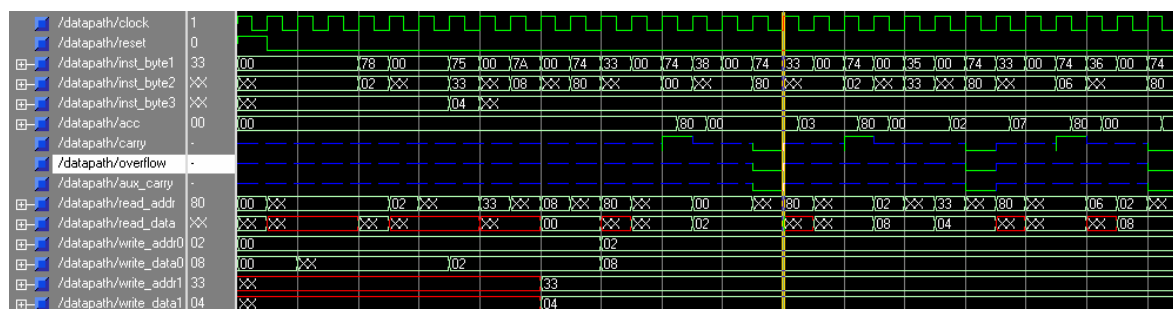
การทำงานส่วนแรกเป็นการกำหนดค่าต่างๆ ให้กับโปรแกรม จากนั้นจะทำการทดสอบการทำคำสั่ง ADDC ระหว่างแอดคิวิตูเลเตอร์กับค่าต่างๆ โดยมีการย้ายข้อมูลเพื่อให้เกิดค่าแฟล็กทคขึ้นภายใน แล้วจึงทำคำสั่ง พบว่าสามารถทำงานได้อย่างถูกต้องในทุกรูปแบบของข้อมูล โดยสังเกตค่าของแอดคิวิตูเลเตอร์ที่ถูกเขียน ดังรูปที่ 6.25 และรูปที่ 6.26 ซึ่งเรียงต่อกันตามลำดับเวลา

```

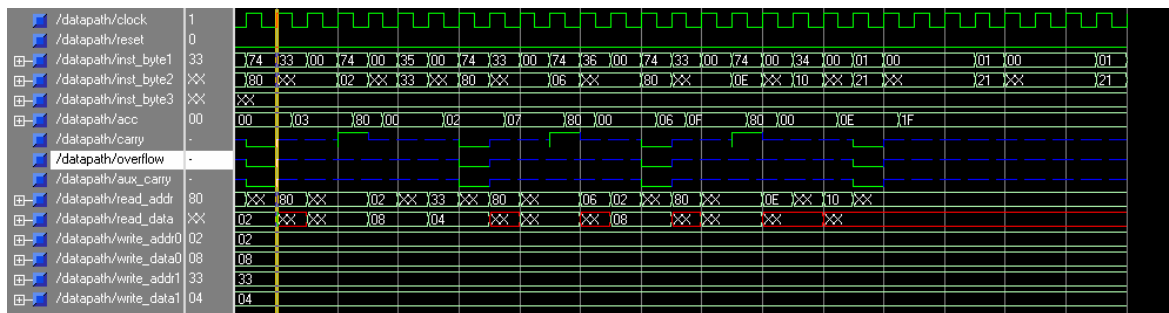
ORG      0
;Basic Data for Test
MOV      R0, #000000010B
MOV      33H, #00000100B
MOV      R2, #00001000B
;Test ADDC Operation - ADDC A, ???
MOV      A, #10000000B
RLC      A                                ;set carry flag for test
MOV      A, #00000000B
ADDC     A, R0
MOV      A, #10000000B
RLC      A                                ;set carry flag for test
MOV      A, #00000010B
ADDC     A, 33H
MOV      A, #10000000B
RLC      A                                ;set carry flag for test
MOV      A, #00000110B
ADDC     A, @R0
MOV      A, #10000000B
RLC      A                                ;set carry flag for test
MOV      A, #00001110B
ADDC     A, #00010000B
MAIN:
AJMP     MAIN
NOP
NOP
NOP
NOP
NOP

```

รูปที่ 6.24 คำสั่งของรอมภายในตัวที่ 11



รูปที่ 6.25 ผลการจำลองการทำงานของรอมภายในตัวที่ 11 ส่วนที่หนึ่ง



รูปที่ 6.26 ผลการจำลองการทำงานของรอมภายในตัวที่ 11 ส่วนที่สอง

6.2.2.12 รอมภายในตัวที่ 12

โปรแกรมของรอมตัวที่ 12 แสดงดังรูปที่ 6.27 เป็นการทำงานของคำสั่ง SUBB ระหว่างแอดคิวมูลเตอร์กับ จากระจิสเตอร์ใช้งานทั่วไป, หน่วยความจำที่อ้างถึงโดยตรง, หน่วยความจำที่อ้างถึงโดยอ้อม และค่าคงที่

```

ORG      0
;Basic Data for Test
MOV      R0, #00000001B
MOV      35H, #00000001B
MOV      R1, #00000001B
;Test SUBB Operation - SUBB A,??? - without carry flag
MOV      A, #00001000B
SUBB     A, R0
MOV      A, #00001000B
SUBB     A, 35H
MOV      A, #00001000B
SUBB     A, @R0
MOV      A, #00001000B
SUBB     A, #00000001B
MAIN:
AJMP     MAIN
NOP
NOP
NOP
NOP
NOP
NOP

```

รูปที่ 6.27 คำสั่งของรอมภายในตัวที่ 12

การทำงานส่วนแรกเป็นการกำหนดค่าต่างๆให้กับโปรแกรม จากนั้นจะทำการทดสอบการทำคำสั่ง SUBB ระหว่างแอดคิวมูลเตอร์กับค่าต่างๆ พบว่าสามารถทำงานได้อย่างถูกต้องในทุกรูปแบบของข้อมูล โดยสังเกตค่าของแอดคิวมูลเตอร์ที่ถูกเขียน ดังรูปที่ 6.28

6.2.2.14 รอมภายในตัวที่ 14

โปรแกรมของรอมตัวที่ 14 แสดงดังรูปที่ 6.32 เป็นการทำงานของคำสั่ง DA, INC และ DEC ซึ่งทำงานกับแอดคิwmูเลเตอร์

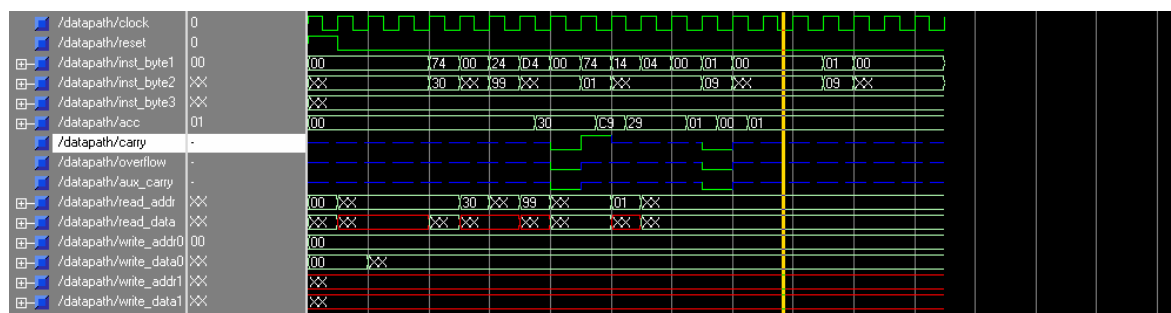
การทำงานส่วนแรกเป็นการกำหนดค่าต่างๆให้กับโปรแกรม โดยการบวกค่าเพื่อให้มีข้อมูลอยู่ในแอดคิwmูเลเตอร์ จากนั้นทำการทดสอบการทำคำสั่ง DA พบว่าสามารถทำงานได้อย่างถูกต้อง โดยสังเกตค่าของแอดคิwmูเลเตอร์ที่ถูกเขียน และการทำคำสั่ง INC และ DEC ก็สามารถตรวจสอบค่าของแอดคิwmูเลเตอร์ได้เช่นกัน ดังรูปที่ 6.33

```

|   ORG      0
|   ;Basic Data for Test
|   MOV      A, #30H
|   ADD      A, #99H
|   ;Test DA Operation
|   DA       A
|   ;Basic Data for Test
|   MOV      A, #0000001B
|   ;Test DEC Operation - DEC A
|   DEC      A                               ;worked
|   ;Test INC Operation - INC A
|   INC      A                               ;worked
|   MAIN:
|   AJMP     MAIN
|   NOP
|   NOP
|   NOP
|   NOP
|   NOP

```

รูปที่ 6.32 คำสั่งของรอมภายในตัวที่ 14



รูปที่ 6.33 ผลการจำลองการทำงานของรอมภายในตัวที่ 14

6.2.2.15 รอมภายในตัวที่ 15

โปรแกรมของรอมตัวที่ 15 แสดงดังรูปที่ 6.34 เป็นการทำงานของคำสั่ง XCH ซึ่งทำงานระหว่างแอดคิwmูเลเตอร์กับค่าต่างๆ เช่น จาก รีจิสเตอร์ใช้งานทั่วไป, หน่วยความจำที่อ้างถึงโดยตรง และหน่วยความจำที่อ้างถึงโดยอ้อม และคำสั่ง XCHD ซึ่งทำงานระหว่างแอดคิwmูเลเตอร์กับหน่วยความจำที่อ้างถึงโดยอ้อม

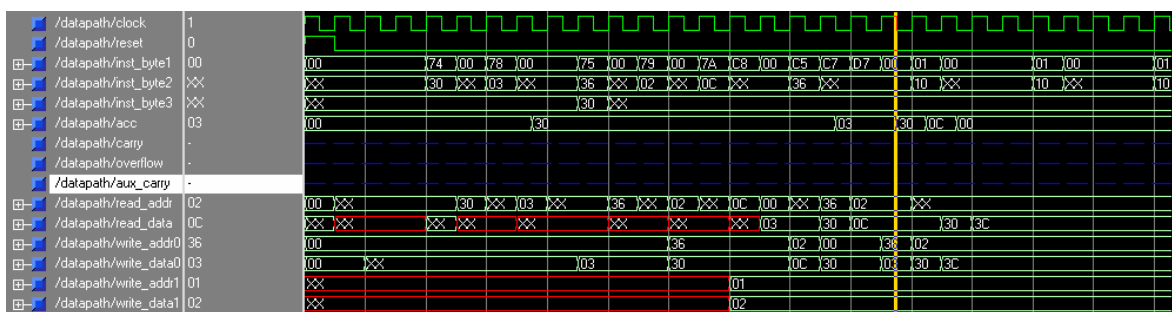
การทำงานส่วนแรกเป็นการกำหนดค่าต่างๆให้กับโปรแกรม จากนั้นทำการทดสอบการทำงานคำสั่งต่างๆว่าสามารถทำงานได้ พบว่าสามารถทำงานได้อย่างถูกต้อง โดยสังเกตค่าของแอดคิwmูเลเตอร์ที่ถูกเขียน และตำแหน่งของหน่วยความจำที่ถูกเขียนพร้อมกับข้อมูลที่เขียนลงไปยังหน่วยความจำตำแหน่งนั้นๆ ดังรูปที่ 6.35

```

ORG      0
;Basic Data for Test
MOV      A, #00110000B
MOV      R0, #00000011B
MOV      36H, #00110000B
MOV      R1, #00000010B
MOV      R2, #00001100B
;Test XCH Operation - XCH A,???
XCH      A, R0
XCH      A, 36H
XCH      A, @R1
;Test XCHD Operation
XCHD     A, @R1
MAIN:
AJMP     MAIN
NOP
NOP
NOP
NOP
NOP

```

รูปที่ 6.34 คำสั่งของรอมภายในตัวที่ 15



รูปที่ 6.35 ผลการจำลองการทำงานของรอมภายในตัวที่ 15

6.3 การบันทึกลงเอฟพีจีเอ

6.3.1 การกำหนดขาสัญญาณเชื่อมต่อ

ในขั้นตอนของการบันทึกโปรแกรมลงเอฟพีจีเอ จำเป็นต้องทำความเข้าใจถึงโครงสร้างการทำงานของเอฟพีจีเอที่นำมาใช้งานว่าสามารถกำหนดการเชื่อมต่อขาให้เข้ากับขาใดของตัวเอฟพีจีเอได้บ้าง โดยทำการศึกษาจากคู่มือการใช้งานของบอร์ดทดลองที่แนบมาด้วยโดยในโครงงานนี้ใช้เอฟพีจีเอตระกูล Spartan 3 ของบริษัท Xilinx เบอร์ XC3S200 โดยใช้บอร์ดทดลอง FPGA-Discovery ของบริษัท ailogictechnology ทำการกำหนดขาต่างๆ ของไมโครคอนโทรลเลอร์ที่ได้ทำการออกแบบกับขาของเอฟพีจีเอซึ่งเชื่อมต่อกับอุปกรณ์ต่างๆบนบอร์ดทดลองไว้ ดังนี้

ขาสัญญาณรีเซ็ตของไมโครคอนโทรลเลอร์ต่อกับสวิตช์กดคิดป้อนยดับที่ขา 44 และเนื่องจากสวิตช์ของบอร์ดทดลองนั้นให้ค่าลอจิกเป็น “1” เมื่อไม่ทำการกด และให้ค่าเป็น “0” เมื่อทำการกด วงจรที่ทำงานจึงรีเซ็ตตัวเองตลอดเวลา ดังนั้นจึงต้องปรับปรุงการเขียนโปรแกรมให้กลับสัญญาณรีเซ็ตที่เข้ามาก่อนเพื่อให้วงจรสามารถทำงานได้อย่างถูกต้อง

ขาสัญญาณนาฬิกาของไมโครคอนโทรลเลอร์ต่อกับคริสตอลจ่ายสัญญาณนาฬิกาซึ่งอยู่บนบอร์ดที่ขา 127 โดยให้สัญญาณนาฬิกาความถี่ 25 เมกะเฮิร์ตซ์

พอร์ต 0 ใช้เป็นอินพุตโดยต่อกับคิพสวิตช์จำนวน 8 ตัว ดังตารางที่ 6.1

บิตที่	ขาของเอฟพีจีเอ
0	52
1	53
2	55
3	56
4	59
5	60
6	63
7	68

ตารางที่ 6.1 การกำหนดขาเชื่อมต่อให้กับพอร์ต 0

พอร์ต์ 1 ใช้เป็นเอาต์พุตโดยต่อกับไดโอดเปล่งแสงจำนวน 8 หลอด ดังตารางที่ 6.2

บิตที่	ขาของเอฟพีจีเอ
0	70
1	77
2	69
3	76
4	74
5	79
6	73
7	78

ตารางที่ 6.2 การกำหนดขาเชื่อมต่อให้กับพอร์ต์ 1

พอร์ต์ 2 ใช้เป็นเอาต์พุตโดยต่อกับตัวแสดงผลเจ็ดส่วนทั้ง 8 บิต ดังตารางที่ 6.3

บิตที่	ขาของเอฟพีจีเอ
0	40
1	35
2	32
3	30
4	27
5	25
6	23
7	20

ตารางที่ 6.3 การกำหนดขาเชื่อมต่อให้กับพอร์ต์ 2

พอร์ต์ 3 ใช้เป็นเอาต์พุตโดยทำการเชื่อมต่อกับขาาร่วมของตัวแสดงผลเจ็ดส่วนทั้ง 4 หลัก ดังตารางที่ 6.4

บิตที่	ขาของเอฟพีจีเอ
0	31

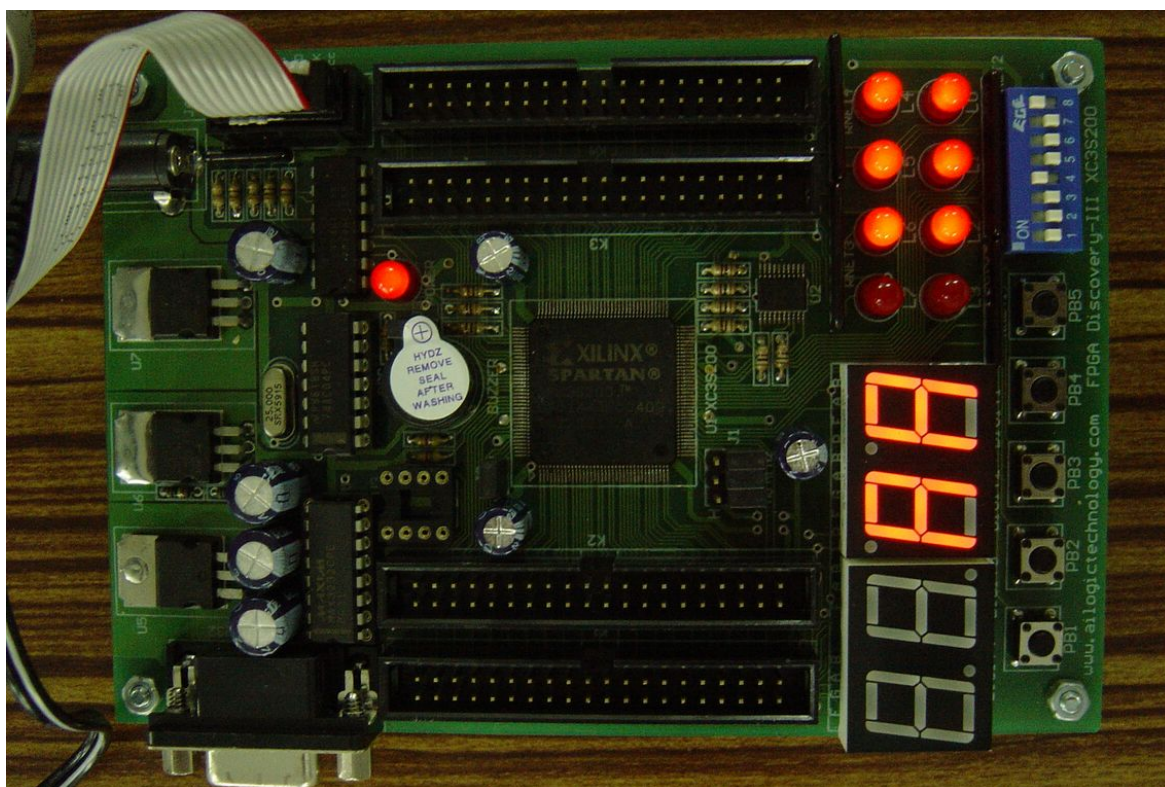
บิตที่	ขาของเอฟพีจีเอ
1	33
2	36
3	41

ตารางที่ 6.4 การกำหนดขาเชื่อมต่อให้กับพอร์ต 3

7.2 การบันทึกลงเอฟพีจีเอและผลการทำงาน

จากนั้นทำการสังเคราะห์วงจร จัดวางและกำหนดการเชื่อมต่อ จากนั้นและทำการบันทึกโดยใช้โปรแกรมดาวน์โหลดข้อมูลผ่านสายดาวน์โหลดข้อมูลลงสู่บอร์ด โดยโปรแกรมที่บันทึกลงไปจะเป็นการอ่านค่าของสวิตช์ที่พอร์ต 0 แล้วนำมาแสดงผลที่พอร์ต 1 และ 2 โดยที่พอร์ต 3 จะกำหนดให้มีการทำงานของตัวแสดงผล 7 ส่วนหลักที่ 3 และ 2

ผลการทำงาน สามารถทำการอ่านค่าจากพอร์ตและนำไปแสดงผลการทำงานได้อย่างถูกต้อง ดังรูปที่ 6.36



รูปที่ 6.36 การทำงานของโปรแกรมที่บันทึกลงบนเอฟพีจีเอ